21.03.2025

Popescu Maria Teodora

345C1

Lab 4 - EP

1. A. pentru un vmstat 1 si stress –cpy $(nproc) –timeout 3s

```
16  0      0 12495060  41936 927648    0    0     0     0   70  269 66  0 34  0  0
16  0      0 12495060  41936 927648    0    0     0     0    5  209 100  0  0  0  0
16  0      0 12495060  41936 927648    0    0     0     0    0  201 100  0  0  0  0
```

```
popi@DESKTOP-91RNSVG:~$ stress --cpu $(nproc) --timeout 3s
stress: info: [1379] dispatching hogs: 16 cpu, 0 io, 0 vm, 0 hdd
stress: info: [1379] successful run completed in 3s
```

pentru vmstat fara stress:

```
nt/cpu/task_02$ vmstat 1
procs -----------memory---------- ---swap-- -----io---- -system-- -
-----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs u
s sy id wa st
 0  0      0 12757928  29744 730372    0    0    21     1    6   20
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   20  149
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   20  142
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   18  132
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   25  152
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   17  133
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   18  136
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   18  130
  0  0 100  0  0
 0  0      0 12757704  29744 730372    0    0     0     0   18  138
  0  0 100  0  0
```

B. ok, deci dintr-un terminal full de comenzi de sistem si date numerice despre ele, se observa ca vmstat deschide “fisiere” adica un fel de procese, apoi citeste din ele, apoi

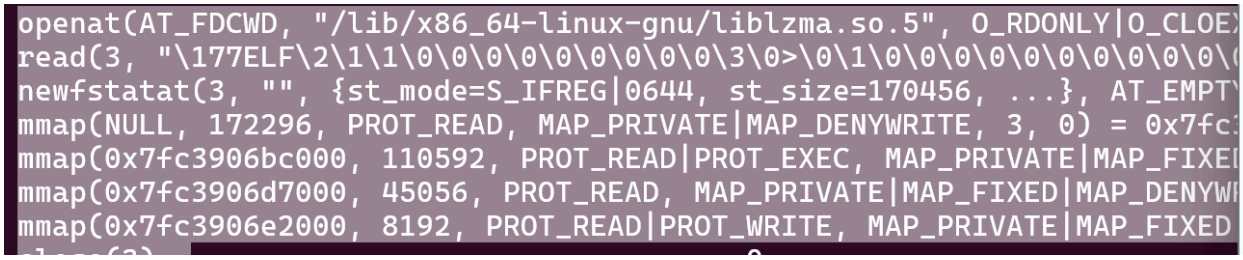
mapeaza memorie; mai face citiri cu lseek. evenatal din cand in cand face newfstatat, apoi inchide fisierul din care citeste metrici. (asta e concluzia la care ajung). si da, toate astea sunt apeluri de sistem.

da, intr-adevar, afiseaza din cand in cand header-ul

write(1, "procs -----------memory---------"..., 81procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----

write(1, " r b   swpd   free   buff cach"..., 81 r b   swpd   free   buff cache   si   so   bi   bo   ….

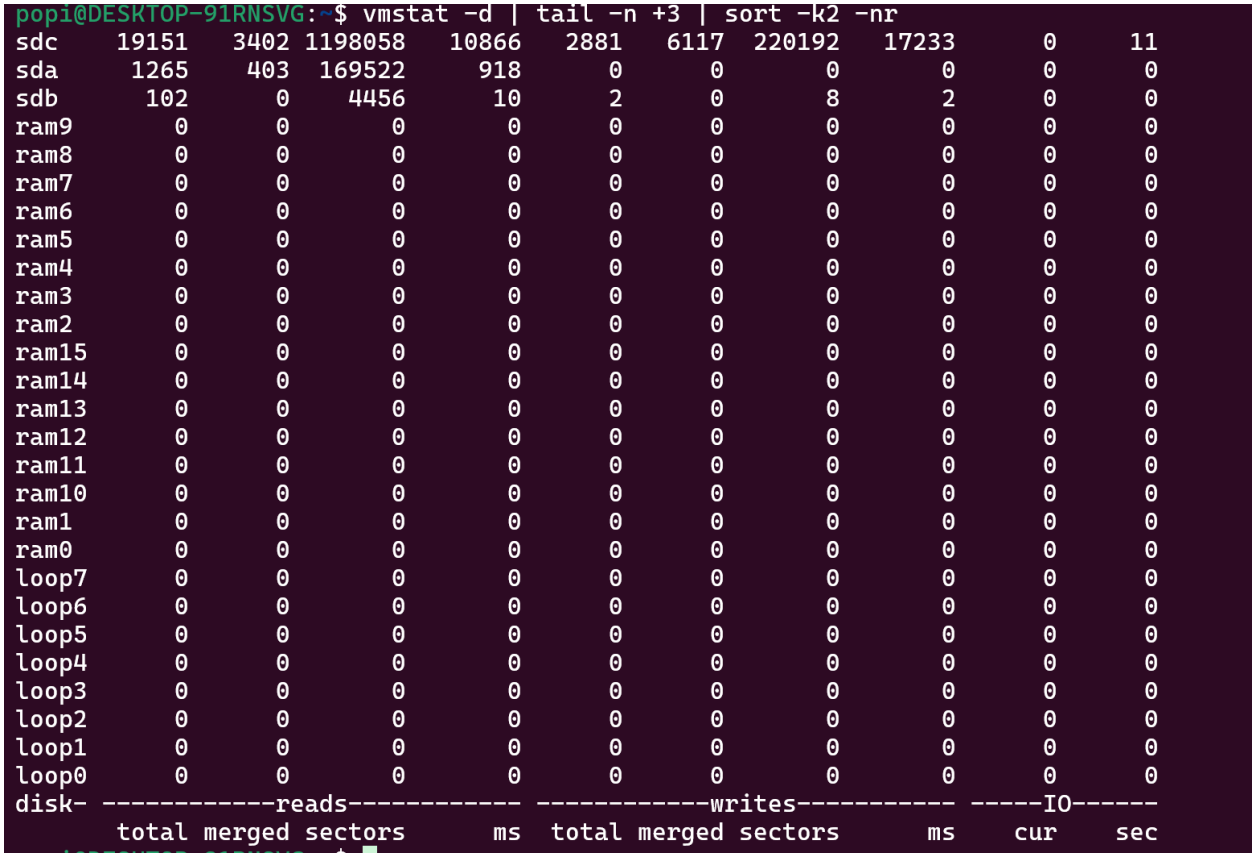
mai jos cateva screenshoturi:



```
close(3)                                = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, (
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
arch_prctl(ARCH_SET_FS, 0x7fc3904547c0) = 0
set_tid_address(0x7fc390454a90)         = 1407
set_robust_list(0x7fc390454aa0, 24)     = 0
rseq(0x7fc390455160, 0x20, 0, 0x53053053) = 0
mprotect(0x7fc3909c1000, 16384, PROT_READ) = 0
mprotect(0x7fc39047d000, 4096, PROT_READ) = 0
mprotect(0x7fc3905b3000, 12288, PROT_READ) = 0
mprotect(0x7fc3905c6000, 4096, PROT_READ) = 0
mprotect(0x7fc3905e6000, 4096, PROT_READ) = 0
mprotect(0x7fc3906b7000, 4096, PROT_READ) = 0
mprotect(0x7fc3906e2000, 4096, PROT_READ) = 0
mprotect(0x7fc3907a2000, 28672, PROT_READ) = 0
mprotect(0x7fc3909e6000, 8192, PROT_READ) = 0
mprotect(0x56213fce0000, 4096, PROT_READ) = 0
mprotect(0x7fc390a51000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_
munmap(0x7fc390a0c000, 41167)           = 0
getrandom("\x01\x47\x9c\xa0\x6f\xce\x6e\x97", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x56214110a000
brk(0x56214112b000)                     = 0x56214112b000
prctl(PR_CAPBSET_READ, CAP_MAC_OVERRIDE) = 1
prctl(PR_CAPBSET_READ, 0x30 /* CAP_??? */) = -1 EINVAL (Invalid argume
prctl(PR_CAPBSET_READ, CAP_CHECKPOINT_RESTORE) = 1
prctl(PR_CAPBSET_READ, 0x2c /* CAP_??? */) = -1 EINVAL (Invalid argume
prctl(PR_CAPBSET_READ, 0x2a /* CAP_??? */) = -1 EINVAL (Invalid argume
prctl(PR_CAPBSET_READ, 0x29 /* CAP_??? */) = -1 EINVAL (Invalid argume
openat(AT_FDCWD, "/proc/self/auxv", O_RDONLY) = 3
newfstatat(3, "", {st_mode=S_IFREG|0400, st_size=0, ...}, AT_EMPTY_PATH
read(3, "!\0\0\0\0\0\0\0 W\240\375\177\0\0003\0\0\0\0\0\0\0000\r\0\0
close(3)                                = 0
openat(AT_FDCWD, "/proc/sys/kernel/osrelease", O_RDONLY) = 3
newfstatat(3, "", {st_mode=S_IFREG|0444, st_size=0, ...}, AT_EMPTY_PATH
read(3, "5.15.133.1-microsoft-standard-WS"..., 1024) = 35
close(3)                                = 0
openat(AT_FDCWD, "/sys/devices/system/cpu/online", O_RDONLY|O_CLOEXEC)
```

```
openat(AT_FDCWD, "/proc/sys/kernel/osrelease", O_RDONLY) = 3
newfstatat(3, "", {st_mode=S_IFREG|0444, st_size=0, ...}, AT_EMPTY_PATH
read(3, "5.15.133.1-microsoft-standard-WS"..., 1024) = 35
close(3)                                = 0
openat(AT_FDCWD, "/proc/meminfo", O_RDONLY) = 3
lseek(3, 0, SEEK_SET)                   = 0
```

C. vmstat -d | tail -n +3 | sort -k2 -nr



mai relevante sunt primele 3 randuri, care afiseaza….

sdc are cel mai mare număr de citiri (1198058), ceea ce indică că acest disc este cel mai utilizat pentru operațiuni de citire.

sda si sdb au mult mai puține citiri comparativ cu sdc.

Discurile RAM (ram0 - ram15) și dispozitivele loop nu au activitate (toate valorile sunt 0). (loopback, de la retea, cred)

Ultima linie este antetul coloanelor și nu face parte din datele sortate.

2. A.

```
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02$ python3 fact_rcrs.py 1000
factorial(1) =
factorial(2) =
factorial(3) =
factorial(4) =
factorial(5) =
factorial(6) =
factorial(7) =
factorial(8) =
factorial(9) =
factorial(10) =
factorial(11) =
factorial(12) =
factorial(13) =
factorial(14) =
factorial(15) =
factorial(16) =
```

```
factorial(998) =
Traceback (most recent call last):
  File "/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02/fact_rcrs.py", line 14, in <module>
    factorial(i)
  File "/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02/fact_rcrs.py", line 7, in factorial
    else n * factorial(n - 1)
  File "/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02/fact_rcrs.py", line 7, in factorial
    else n * factorial(n - 1)
  File "/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02/fact_rcrs.py", line 7, in factorial
    else n * factorial(n - 1)
  [Previous line repeated 995 more times]
  File "/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_02/fact_rcrs.py", line 6, in factorial
    return 1 if n == 0 \
RecursionError: maximum recursion depth exceeded in comparison
```

eroarea de mai sus este normala caci linuxul are in cel ceva care il face sa nu mearga la infinit sau sa nu foloseasca foarte multe resurse la apelurile recurseive care sunt consumatoare de procese, memorie, timp etc.

trebuie sa modific scriptul

am facut conform a ceea ce e indicat pe ocw, am pus sys.setrecursion sth la inceput, dupa import

```python
#!/usr/bin/python3

import sys

# Creștem limita de recursie pentru a evita crash-ul
sys.setrecursionlimit(10**6)  # Setează o limită mai mare, dar rezonabilă

def factorial(n):
    return 1 if n == 0 else n * factorial(n - 1)

if __name__ == '__main__':
    big_number = int(sys.argv[1])

    for i in range(1, big_number + 1):
```

```
        print(f'factorial({i}) = {factorial(i)}')
```

si gen imi afiseaza si rezultatele pe parcurs, nu se mai opreste, nu mai da eroare

factorial(1000) =

4023872600770937735437024339230039857193748642107146325437999104299385123986290205920442084869694048004799886101971960586316668729948085589013238296699445909974245040870737599188236277271887325197795059509952761208749754624970436014182780946464962910563938874378864873371191810458257836478499770124766328898 35 …. (ceva numar mare)

B. pentru argumentul 10000, pe soluria anterioara se opreste la 1558

idle:

09:12:27	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
09:12:28	all	0.00	0.00	0.06	0.06	0.00	0.00	0.00	0.00	0.00	99.87
09:12:28	0	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	99.00
09:12:28	1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	2	0.00	0.00	0.99	0.00	0.00	0.00	0.00	0.00	0.00	99.01
09:12:28	3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	4	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	5	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	6	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	8	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	12	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	13	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	14	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:28	15	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00

apoi se ved ca core-ul 4 e folosit, daca rulez scriptul

09:12:26	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
09:12:27	all	1.69	0.00	0.25	0.00	0.00	0.00	0.00	0.00	0.00	98.06
09:12:27	0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	1	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	99.00
09:12:27	2	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	4	26.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	73.00
09:12:27	5	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	6	0.00	0.00	0.99	0.00	0.00	0.00	0.00	0.00	0.00	99.01
09:12:27	7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	8	0.00	0.00	0.99	0.00	0.00	0.00	0.00	0.00	0.00	99.01
09:12:27	9	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	10	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	11	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	12	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	99.00
09:12:27	13	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	14	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
09:12:27	15	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00

stress --cpu $(($(nproc) - 1))

taskset -c 0 python3 fact_rcrs.py 10000

mpstat -P ALL 1

```
Linux 5.15.133.1-microsoft-standard-WSL2 (DESKTOP-91RNSVG)      03/21/25        _x86_64_        (16 C

09:15:22      CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
09:15:23      all   92.82    0.00    0.00    0.00    0.00    0.99    0.00    0.00    0.00    6.19
09:15:23        0  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        1  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        2  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        3  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        4  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        5  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        6  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        7  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        8  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23        9  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23       10  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23       11  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23       12    0.00    0.00    0.00    0.00    0.00   13.79    0.00    0.00    0.00   86.21
09:15:23       13  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23       14  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:23       15  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
```

asta ^ e fara aia cu pus doar pe un core

—----------

apoi asta de mai jos e cand rulez scriptul pe core0

```
09:15:30      CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
09:15:31      all   95.89    0.00    0.12    0.00    0.00    0.44    0.00    0.00    0.00    3.55
09:15:31        0   37.14    0.00    1.90    0.00    0.00    6.67    0.00    0.00    0.00   54.29
09:15:31        1  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        2  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        3  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        4  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        5  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        6  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        7  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        8  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31        9  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       10  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       11  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       12  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       13  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       14  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:15:31       15  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
```

aici, mai jos, e imediat dupa ce s-a terminat de rulat scriptul

```
Average:      CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
Average:      all   94.40    0.00    0.17    0.00    0.00    0.68    0.00    0.00    0.00    4.75
Average:        0   80.05    0.00    2.01    0.00    0.00    2.12    0.00    0.00    0.00   15.82
Average:        1  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        2   99.94    0.00    0.06    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        3  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        4  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        5  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        6  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        7  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        8  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:        9  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:       10  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:       11  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:       12   35.87    0.00    0.56    0.00    0.00    8.11    0.00    0.00    0.00   55.46
Average:       13  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:       14  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
Average:       15  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
```

C. taskset -c $(seq 1 2 $(($(nproc) - 1)) | paste -sd,) stress --cpu $(($(nproc) / 2))

Generez lista de nuclee impare *(1, 3, 5, ...)*. cu seq sa fac o secvnebta cu numere consecutive, din care iau din 2 in 2 incepand cu prima

Formatez sirul de mai sus într-un șir separat prin virgulă *(1,3,5,7,...)*. (paste sd,)

Leagă procesul stress doar pe aceste nuclee *(cu taskset -c)*, date ca argument (gen outputul il dau ca input)

Rulez stress cu un număr de lucrători egal cu jumătate din totalul nucleelor.

```
popi@DESKTOP-91RNSVG:~$ mpstat -P ALL 1
Linux 5.15.133.1-microsoft-standard-WSL2 (DESKTOP-91RNSVG)      03/21/25        _x86_64_        (16 CPU)

09:27:54      CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
09:27:55      all   49.97    0.00    0.00    0.00    0.00    0.06    0.00    0.00    0.00   49.97
09:27:55        0    0.00    0.00    0.00    0.00    0.00    0.99    0.00    0.00    0.00   99.01
09:27:55        1  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55        2    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55        3  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55        4    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55        5  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55        6    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55        7  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55        8    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55        9  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55       10    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55       11  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55       12    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55       13  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
09:27:55       14    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00  100.00
09:27:55       15  100.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00    0.00
```

```python
import os
import time
import zipfile
import matplotlib.pyplot as plt
import numpy as np

# selectez fisierele
```

```
fisiereeee = ["big.txt", "marbles.bmp", "ray.bmp"]

# verific sa vad daca exista, sa nu am eroare
for file in fisiereeee:
    if not os.path.exists(file):
        print(f"Ooof, nu exista")
        exit(1)

# vreau sa pun rezultatele compresiei undeva ca sa fac grafice
compression_levels = list(range(10))  # Zip levels 0-9
compression_ratios = []
duratas = []

# incerc de mai multe ori
incercari = 3

for niv in compression_levels:
    avg_ratio = []
    avg_time = []

    for _ in range(incercari):
        # il fac zip
        nume = f"test_{niv}.zip"
        start = time.time()

        with zipfile.ZipFile(nume, 'w', compression=zipfile.ZIP_DEFLATED,
compresslevel=niv) as zipf:
            for file in fisiereeee:
                zipf.write(file, os.path.basename(file))

        end = time.time()
        durata = end - start

        # aflu dimensiunile
        original_size = sum(os.path.getsize(file) for file in fisiereeee)
        compressed_size = os.path.getsize(nume)

        # calculez rata de compresie
        compression_ratio = (compressed_size / original_size) * 100
```

```
        avg_ratio.append(compression_ratio)
        avg_time.append(durata)

        os.remove(nume)

    # salvez rezultatele
    compression_ratios.append(np.mean(avg_ratio))
    duratas.append(np.mean(avg_time))

# fac grafice
plt.figure(figsize=(10, 5))

# rata de compresie
plt.subplot(1, 2, 1)
plt.plot(compression_levels, compression_ratios, marker='o',
label="Compression Ratio (%)")
plt.xlabel("Zip Compression Level")
plt.ylabel("Compression Ratio (%)")
plt.title("Compression Ratio vs Zip Level")
plt.grid(True)
plt.legend()

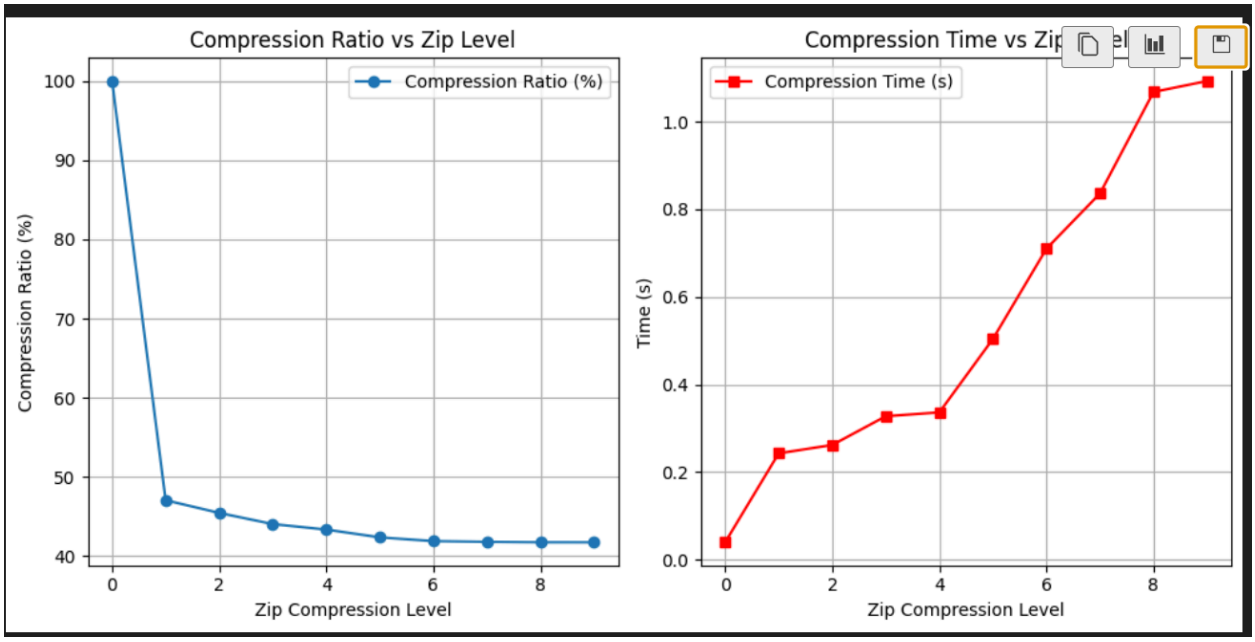
# timp de compresie
plt.subplot(1, 2, 2)
plt.plot(compression_levels, duratas, marker='s', label="Compression Time
(s)", color="red")
plt.xlabel("Zip Compression Level")
plt.ylabel("Time (s)")
plt.title("Compression Time vs Zip Level")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

3.

Scriptul selectează mai multe fișiere mari pentru testare prezente in schelet. Apoi, parcurge toate nivelurile de compresie de la 0 la 9, măsurând timpul necesar pentru comprimare și dimensiunea fișierului comprimat. Pe baza acestor date, calculează raportul de compresie (dimensiunea

comprimată / dimensiunea originală * 100). Pentru rezultate mai precise, fiecare test este repetat de 3 ori, iar valorile sunt mediate. În final, scriptul generează două grafice: unul care arată raportul de compresie în funcție de nivelul de compresie, și altul care ilustrează timpul de compresie pentru fiecare nivel. Acest lucru ajută la analizarea eficienței fiecărui nivel de compresie ZIP.



Primul grafic:

Nivelul 0 (fără compresie) are un raport de 100%, ceea ce înseamnă că fișierul nu a fost comprimat deloc (doar împachetat).

Nivelurile 1-3 aduc o scădere semnificativă a dimensiunii fișierului.

După nivelul 4, eficiența compresiei începe să se stabilizeze (~40-45%).

Nivelurile 6-9 au o compresie aproape identică, ceea ce sugerează diminishing returns (beneficii marginale reduse).

Nivelul 6 este cel optim (default) deoarece oferă o bună balanță între compresie și timp.

Nivelurile 8-9 nu aduc beneficii majore, dar cresc timpul necesar pentru comprimare.

Al doilea grafic:

Nivelul 0 este cel mai rapid, deoarece nu comprimă, doar împachetează.

Timpul crește progresiv odată cu creșterea nivelului de compresie.

După nivelul 6, timpul de compresie crește accelerat, fără o îmbunătățire semnificativă a raportului de compresie.

Nivelurile 8-9 sunt cele mai lente, necesitând cel mai mult timp pentru o îmbunătățire minoră a dimensiunii fișierului.
Nivelurile mici (1-3) sunt rapide și oferă o compresie decentă.
Nivelul 6 este cel mai echilibrat între compresie și viteză.
Nivelurile 8-9 nu sunt eficiente, deoarece cresc drastic timpul de compresie fără a reduce semnificativ dimensiunea fișierului.

4.

A. setup -> nimic concludent, instalat clang, transformat in cod assembly:
   sudo apt install clang && clang -S -masm=intel csum.c -o csum.s && cat csum.s
   atat codul in C, cat si cel in assembly, nu par foarte sofisticate, desi, se intampla diverse lucruri

   pentru a observa ce se intampla, am adaugat un fel de breakpoint uri cu volatile in cod:
   // antet functie
   asm volatile("# LLVM-MCA-BEGIN" ::: "memory");
   // Codul functiei
   asm volatile("# LLVM-MCA-END" ::: "memory");
   // return

   si am apelat asa
   clang -S -O2 -masm=intel csum.c -o csum_opt.s

   si apoi am incercat si un llvm pentru statistici

   llvm-mca -mtriple=x86_64 -mcpu=skylake -iterations=100 -bottleneck-analysis csum.s

```
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_04$ llvm-mca -mtriple=x86_64 -mcpu=skylake -iterations=100 -bottleneck-analysis c
sum.s

[0] Code Region

Iterations:        100
Instructions:      8100
Total Cycles:      1854
Total uOps:        8100

Dispatch Width:    6
uOps Per Cycle:    4.37
IPC:               4.37
Block RThroughput: 14.0


Cycles with backend pressure increase [ 74.65% ]
Throughput Bottlenecks:
  Resource Pressure       [ 71.63% ]
  - SKLPort0  [ 44.50% ]
  - SKLPort1  [ 46.49% ]
  - SKLPort2  [ 3.02% ]
  - SKLPort3  [ 3.02% ]
  - SKLPort5  [ 67.42% ]
  - SKLPort6  [ 44.50% ]
  Data Dependencies:      [ 31.98% ]
  - Register Dependencies [ 31.98% ]
  - Memory Dependencies   [ 0.00% ]

Critical sequence based on the simulation:

              Instruction                                 Dependency Information
 +----< 66.   add       rdx, -2
 |
 |    < loop carried >
 |
 +----> 0.    cmp       rdx, 2                           ## REGISTER dependency:  rdx
 |      1.    jb        .LBB0_10
```

```
Critical sequence based on the simulation:

              Instruction                                 Dependency Information
 +----< 66.   add       rdx, -2
 |
 |    < loop carried >
 |
 +----> 0.    cmp       rdx, 2                           ## REGISTER dependency:  rdx
 |      1.    jb        .LBB0_10
 +----> 2.    lea       rcx, [rdx - 2]                   ## RESOURCE interference:  SKLPort5 [ probability: 7% ]
 +----> 3.    cmp       rcx, 6                           ## REGISTER dependency:  rcx
 +----> 4.    jb        .LBB0_9                                 ## REGISTER dependency:  flags
 +----> 5.    shr       rcx                              ## RESOURCE interference:  SKLPort0 [ probability: 22% ]
 +----> 6.    add       rcx, 1                           ## REGISTER dependency:  rcx
 +----> 7.    mov       r8, rcx                          ## REGISTER dependency:  rcx
 +----> 8.    and       r8, -4                           ## REGISTER dependency:  r8
 |      9.    movq      xmm0, rax
 +----> 10.   lea       rax, [r8 - 4]                    ## REGISTER dependency:  r8
 +----> 11.   mov       r9, rax                          ## REGISTER dependency:  rax
 +----> 12.   shr       r9, 2                            ## REGISTER dependency:  r9
 +----> 13.   add       r9, 1                            ## REGISTER dependency:  r9
 |      14.   test      rax, rax
 |      15.   je        .LBB0_3
 +----> 16.   mov       rax, r9                          ## REGISTER dependency:  r9
 +----> 17.   and       rax, -2                          ## REGISTER dependency:  rax
 |      18.   pxor      xmm2, xmm2
 |      19.   xor       edi, edi
 |      20.   pxor      xmm1, xmm1
 |      21.   movd      xmm3, dword ptr [rsi + 2*rdi]
 |      22.   movd      xmm4, dword ptr [rsi + 2*rdi + 4]
 |      23.   punpcklwd xmm3, xmm2
 |      24.   punpckldq xmm3, xmm2
 |      25.   paddq     xmm3, xmm0
 |      26.   punpcklwd xmm4, xmm2
 |      27.   punpckldq xmm4, xmm2
 |      28.   paddq     xmm4, xmm1
 |      29.   movd      xmm0, dword ptr [rsi + 2*rdi + 8]
 |      30.   movd      xmm1, dword ptr [rsi + 2*rdi + 12]
 |      31.   punpcklwd xmm0, xmm2
```

```
Instruction Info:
[1]: #uOps
[2]: Latency
[3]: RThroughput
[4]: MayLoad
[5]: MayStore
[6]: HasSideEffects (U)

[1]    [2]    [3]    [4]    [5]    [6]    Instructions:
 1      1     0.25                        cmp   rdx, 2
 1      1     0.50                        jb    .LBB0_10
 1      1     0.50                        lea   rcx, [rdx - 2]
 1      1     0.25                        cmp   rcx, 6
 1      1     0.50                        jb    .LBB0_9
 1      1     0.50                        shr   rcx
 1      1     0.25                        add   rcx, 1
 1      1     0.25                        mov   r8, rcx
 1      1     0.25                        and   r8, -4
 1      1     1.00                        movq  xmm0, rax
 1      1     0.50                        lea   rax, [r8 - 4]
 1      1     0.25                        mov   r9, rax
 1      1     0.50                        shr   r9, 2
 1      1     0.25                        add   r9, 1
 1      1     0.25                        test  rax, rax
 1      1     0.50                        je    .LBB0_3
 1      1     0.25                        mov   rax, r9
 1      1     0.25                        and   rax, -2
 1      0     0.17                        pxor  xmm2, xmm2
 1      0     0.17                        xor   edi, edi
 1      0     0.17                        pxor  xmm1, xmm1
 1      5     0.50    *                   movd  xmm3, dword ptr [rsi + 2*rdi]
 1      5     0.50    *                   movd  xmm4, dword ptr [rsi + 2*rdi + 4]
 1      1     1.00                        punpcklwd     xmm3, xmm2
 1      1     1.00                        punpckldq     xmm3, xmm2
 1      1     0.33                        paddq xmm3, xmm0
 1      1     1.00                        punpcklwd     xmm4, xmm2
 1      1     1.00                        punpckldq     xmm4, xmm2
 1      1     0.33                        paddq xmm4, xmm1
 1      5     0.50    *                   movd  xmm0, dword ptr [rsi + 2*rdi + 8]
 1      5     0.50    *                   movd  xmm1, dword ptr [rsi + 2*rdi + 12]
 1      1     1.00                        punpcklwd     xmm0, xmm2
```

```
Resource pressure per iteration:
[0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]
 -      -     16.57  16.28  4.00   4.00    -     17.87  16.28   -

Resource pressure by instruction:
[0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    Instructions:
 -      -     0.14   0.71    -      -      -     0.07   0.08    -     cmp       rdx, 2
 -      -     0.31    -      -      -      -      -     0.69    -     jb        .LBB0_10
 -      -      -     0.85    -      -      -     0.15    -      -     lea       rcx, [rdx - 2]
 -      -     0.20   0.44    -      -      -     0.21   0.15    -     cmp       rcx, 6
 -      -     0.44    -      -      -      -      -     0.56    -     jb        .LBB0_9
 -      -     0.36    -      -      -      -      -     0.64    -     shr       rcx
 -      -     0.41   0.43    -      -      -     0.07   0.09    -     add       rcx, 1
 -      -     0.21   0.42    -      -      -     0.21   0.16    -     mov       r8, rcx
 -      -     0.21   0.35    -      -      -     0.01   0.43    -     and       r8, -4
 -      -      -      -      -      -      -     1.00    -      -     movq      xmm0, rax
 -      -      -     0.71    -      -      -     0.29    -      -     lea       rax, [r8 - 4]
 -      -     0.23   0.14    -      -      -     0.14   0.49    -     mov       r9, rax
```

din screenshoturi, se intelege ca:

- Unele instrucțiuni x86 sunt descompuse în mai multe micro-operații.
- câte micro-operații sunt asociate fiecărei instrucțiuni.
- Porturile CPU sunt unități funcționale care se ocupă de anumite tipuri de instrucțiuni:
  - Port 0,1 → Instrucțiuni aritmetice.
  - Port 2,3 → Operații de încărcare (load).
  - Port 4 → Operații de stocare (store).
  - Port 5 → Operații vectoriale.
  - Port 6 → Instrucțiuni întregi și salturi (branching).
  - Port 7 → Generare de adrese (AGU - Address Generation Unit).
  - un port este supraîncărcat înseamnă că acel tip de instrucțiuni poate fi un bottleneck.
- se poate calcula throughput-ul estimat caci acest tool arata cât de rapid se poate rula secvența de cod, estimând câte cicluri de ceas sunt necesare.
- alte date: dependinte, latenta etc

Codul a fost analizat pe 100 de iterații.

Dispatch Width: 6 → Numărul maxim de instrucțiuni care pot fi emise simultan de CPU.

Procesorul pare să ruleze eficient codul, dar există spațiu pentru optimizări.
74.65% backend pressure → Codul are o presiune/stress semnificativă pe unitățile de execuție ale CPU. -> Porturile 0 și 1 sunt utilizate intensiv, ceea ce indică multe operații aritmetice, in timp ce portul 5 (folosit pentru vectorizare) are cea mai mare presiune → probabil că unele instrucțiuni sunt folosite intens, iar optimizarea lor ar putea ajuta.

instrucțiunile cu cele mai mari dependențe pe porturi: add, cmp, jb etc -> Problema majoră: Dependență între rdx și rcx => Instrucțiunile trebuie să aștepte finalizarea operației anterioare → execuție mai lentă =>
SOLUTII: Evitarea mutărilor inutile între registre.

Reducerea accesului la memorie → Folosirea registrelor ori de câte ori este posibil.

C.

După rulare, llvm-mca va afișa instrucțiunile problematice, cu cauzele potențiale ale blocajelor:

Port 5 este cel mai solicitat → utilizarea masivă a instrucțiunilor vectoriale (AVX/SSE) poate cauza un bottleneck.

Port 6 (instrucțiuni întregi și salturi - branching) → Indică o posibilă dependență pe ramurile execuției.

Dependența pe registrul rdx

- add rdx, -2 → Instrucțiunea cmp rdx, 2 trebuie să aștepte modificarea lui rdx.
- Aceasta încetinește execuția, deoarece CPU trebuie să finalizeze add înainte de cmp.

SOLUTIE: Folosirea unui registru auxiliar, de explu, mai jos:

mov rax, rdx

add rax, -2

cmp rax, 2

Instrucțiuni problematice → cmp, jb, lea, mov, and, shr, add

Porturile cele mai afectate:

- Port 2 și 3 (load unit) → acces intens la memorie.
- Port 5 (vectorizare AVX/SSE) → congestie pe unitățile SIMD.
- Port 6 (integer ALU și branching) → probleme la ramurile de execuție.

Reducerea accesului la memorie → Folosirea mai eficientă a registrelor pentru a evita încărcările frecvente.

```
popi@DESKTOP-91RNSVG:/mnt/d/fac/an4/sem2/ep/labs/lab3/EP-labs/content/cpu/task_04$ llvm-mca -mtriple=x86_64 -mcpu=skylake -iterations=100 -bottleneck-analysis c
sum.s

[0] Code Region

Iterations:        100
Instructions:      8100
Total Cycles:      1854
Total uOps:        8100

Dispatch Width:    6
uOps Per Cycle:    4.37
IPC:               4.37
Block RThroughput: 14.0


Cycles with backend pressure increase [ 74.65% ]
Throughput Bottlenecks:
  Resource Pressure       [ 71.63% ]
  - SKLPort0  [ 44.50% ]
  - SKLPort1  [ 46.49% ]
  - SKLPort2  [ 3.02% ]
  - SKLPort3  [ 3.02% ]
  - SKLPort5  [ 67.42% ]
  - SKLPort6  [ 44.50% ]
  Data Dependencies:      [ 31.98% ]
  - Register Dependencies [ 31.98% ]
  - Memory Dependencies   [ 0.00% ]

Critical sequence based on the simulation:

              Instruction                                 Dependency Information
 +----< 66.   add       rdx, -2
 |
 |    < loop carried >
 |
 +----> 0.    cmp       rdx, 2                           ## REGISTER dependency:  rdx
```

```
Critical sequence based on the simulation:

              Instruction                                 Dependency Information
 +----< 66.   add       rdx, -2
 |
 |    < loop carried >
 |
 +----> 0.    cmp       rdx, 2                           ## REGISTER dependency:  rdx
 |      1.    jb        .LBB0_10
 +----> 2.    lea       rcx, [rdx - 2]                   ## RESOURCE interference:  SKLPort5 [ probability: 7% ]
 +----> 3.    cmp       rcx, 6                           ## REGISTER dependency:  rcx
 +----> 4.    jb        .LBB0_9                                ## REGISTER dependency:  flags
 +----> 5.    shr       rcx                              ## RESOURCE interference:  SKLPort0 [ probability: 22% ]
 +----> 6.    add       rcx, 1                           ## REGISTER dependency:  rcx
 +----> 7.    mov       r8, rcx                          ## REGISTER dependency:  rcx
 +----> 8.    and       r8, -4                           ## REGISTER dependency:  r8
 |      9.    movq      xmm0, rax
 +----> 10.   lea       rax, [r8 - 4]                    ## REGISTER dependency:  r8
 +----> 11.   mov       r9, rax                          ## REGISTER dependency:  rax
 +----> 12.   shr       r9, 2                            ## REGISTER dependency:  r9
 +----> 13.   add       r9, 1                            ## REGISTER dependency:  r9
 |      14.   test      rax, rax
 |      15.   je        .LBB0_3
 +----> 16.   mov       rax, r9                          ## REGISTER dependency:  r9
 +----> 17.   and       rax, -2                          ## REGISTER dependency:  rax
 |      18.   pxor      xmm2, xmm2
 |      19.   xor       edi, edi
 |      20.   pxor      xmm1, xmm1
 |      21.   movd      xmm3, dword ptr [rsi + 2*rdi]
 |      22.   movd      xmm4, dword ptr [rsi + 2*rdi + 4]
 |      23.   punpcklwd xmm3, xmm2
 |      24.   punpckldq xmm3, xmm2
```

```
|      43.   lea       rax, [r8 + r8]
+----> 44.   test      r9b, 1                          ## RESOURCE interference:  SKLPort0 [ probability: 21% ]
+----> 45.   je        .LBB0_8                                 ## REGISTER dependency:  flags
|      46.   movd      xmm2, dword ptr [rsi + 2*rdi]
|      47.   movd      xmm3, dword ptr [rsi + 2*rdi + 4]
|      48.   pxor      xmm4, xmm4
|      49.   punpcklwd xmm2, xmm4
|      50.   punpckldq xmm2, xmm4
|      51.   paddq     xmm0, xmm2
|      52.   punpcklwd xmm3, xmm4
|      53.   punpckldq xmm3, xmm4
|      54.   paddq     xmm1, xmm3
|      55.   paddq     xmm0, xmm1
|      56.   sub       rdx, rax
|      57.   lea       rsi, [rsi + 2*r8]
|      58.   pshufd    xmm1, xmm0, 238
|      59.   paddq     xmm1, xmm0
|      60.   movq      rax, xmm1
|      61.   cmp       rcx, r8
|      62.   je        .LBB0_10
|      63.   movzx     ecx, word ptr [rsi]
+----> 64.   add       rsi, 2                          ## RESOURCE interference:  SKLPort6 [ probability: 22% ]
|      65.   add       rax, rcx
|      66.   add       rdx, -2
|      67.   cmp       rdx, 1
```

```
Instruction Info:
[1]: #uOps
[2]: Latency
[3]: RThroughput
[4]: MayLoad
[5]: MayStore
[6]: HasSideEffects (U)

[1]    [2]    [3]    [4]    [5]    [6]    Instructions:
 1      1     0.25                        cmp   rdx, 2
 1      1     0.50                        jb    .LBB0_10
 1      1     0.50                        lea   rcx, [rdx - 2]
 1      1     0.25                        cmp   rcx, 6
 1      1     0.50                        jb    .LBB0_9
 1      1     0.50                        shr   rcx
 1      1     0.25                        add   rcx, 1
 1      1     0.25                        mov   r8, rcx
 1      1     0.25                        and   r8, -4
 1      1     1.00                        movq  xmm0, rax
 1      1     0.50                        lea   rax, [r8 - 4]
 1      1     0.25                        mov   r9, rax
 1      1     0.50                        shr   r9, 2
 1      1     0.25                        add   r9, 1
 1      1     0.25                        test  rax, rax
 1      1     0.50                        je    .LBB0_3
 1      1     0.25                        mov   rax, r9
 1      1     0.25                        and   rax, -2
 1      0     0.17                        pxor  xmm2, xmm2
 1      0     0.17                        xor   edi, edi
 1      0     0.17                        pxor  xmm1, xmm1
 1      5     0.50    *                   movd  xmm3, dword ptr [rsi + 2*rdi]
 1      5     0.50    *                   movd  xmm4, dword ptr [rsi + 2*rdi + 4]
 1      1     1.00                        punpcklwd     xmm3, xmm2
 1      1     1.00                        punpckldq     xmm3, xmm2
```

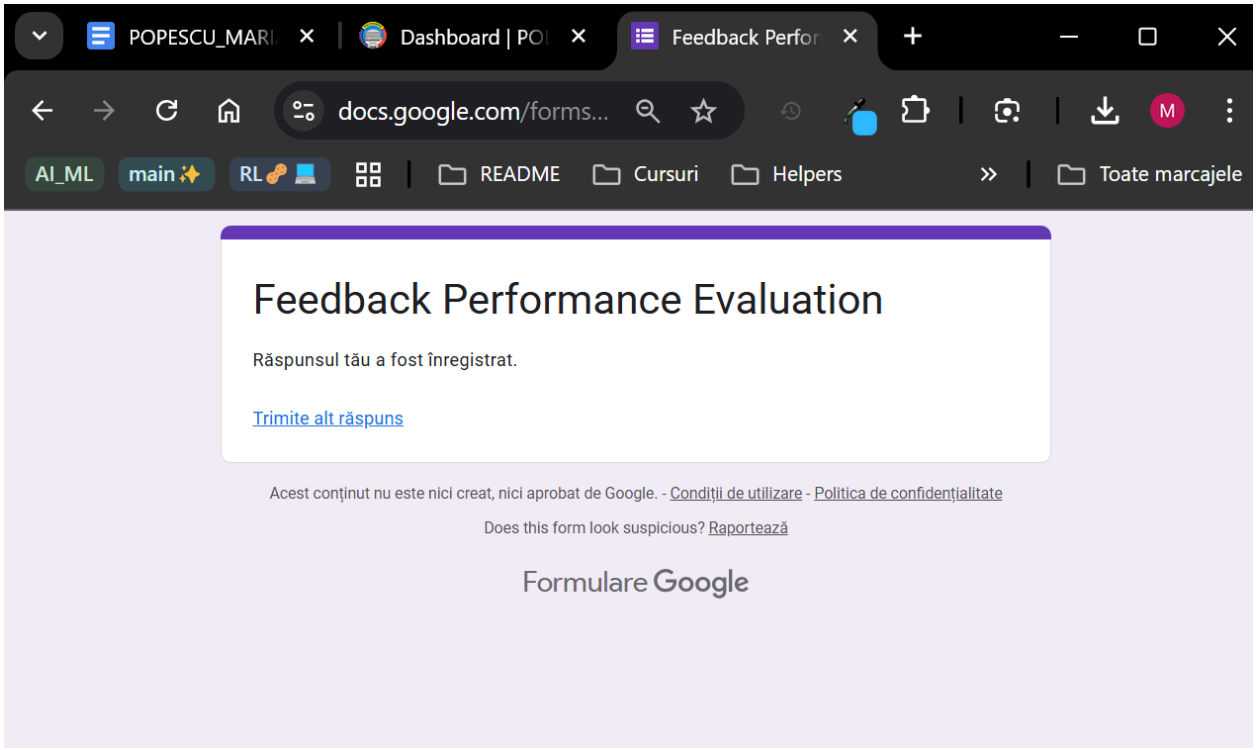
```
Resources:
[0]   - SKLDivider
[1]   - SKLFPDivider
[2]   - SKLPort0
[3]   - SKLPort1
[4]   - SKLPort2
[5]   - SKLPort3
[6]   - SKLPort4
[7]   - SKLPort5
[8]   - SKLPort6
[9]   - SKLPort7


Resource pressure per iteration:
[0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]
 -      -     16.57  16.28  4.00   4.00    -     17.87  16.28   -

Resource pressure by instruction:
[0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    Instructions:
 -      -     0.14   0.71    -      -      -     0.07   0.08    -     cmp       rdx, 2
 -      -     0.31    -      -      -      -      -     0.69    -     jb        .LBB0_10
 -      -      -     0.85    -      -      -     0.15    -      -     lea       rcx, [rdx - 2]
 -      -     0.20   0.44    -      -      -     0.21   0.15    -     cmp       rcx, 6
 -      -     0.44    -      -      -      -      -     0.56    -     jb        .LBB0_9
 -      -     0.36    -      -      -      -      -     0.64    -     shr       rcx
 -      -     0.41   0.43    -      -      -     0.07   0.09    -     add       rcx, 1
 -      -     0.21   0.42    -      -      -     0.21   0.16    -     mov       r8, rcx
 -      -     0.21   0.35    -      -      -     0.01   0.43    -     and       r8, -4
 -      -      -      -      -      -      -     1.00    -      -     movq      xmm0, rax
 -      -      -     0.71    -      -      -     0.29    -      -     lea       rax, [r8 - 4]
 -      -     0.23   0.14    -      -      -     0.14   0.49    -     mov       r9, rax
 -      -     0.16    -      -      -      -      -     0.84    -     shr       r9, 2
 -      -     0.21   0.07    -      -      -     0.08   0.64    -     add       r9, 1
 -      -     0.21   0.14    -      -      -     0.42   0.23    -     test      rax, rax
 -      -     0.63    -      -      -      -      -     0.37    -     je        .LBB0_3
 -      -     0.07   0.29    -      -      -     0.28   0.36    -     mov       rax, r9
 -      -     0.28   0.21    -      -      -     0.07   0.44    -     and       rax, -2
 -      -      -      -      -      -      -      -      -      -     pxor      xmm2, xmm2
```

.

Feedback:


POPESCU_MARI
Dashboard | POI
Feedback Perfor
docs.google.com/forms...
AI_ML
main ✨
RL 🥜 💻
README
Cursuri
Helpers
Toate marcajele
Feedback Performance Evaluation
Răspunsul tău a fost înregistrat.
Trimite alt răspuns
Acest conținut nu este nici creat, nici aprobat de Google. - Condiții de utilizare - Politica de confidențialitate
Does this form look suspicious? Raportează
Formulare Google